T00 · REVISIONE R6

Dal codice al calcolo scientifico

Fondamenti operativi in R e confronto con Python

33 slide · teoria, derivazioni, figure e applicazioni in R.
Provenienza dei dati e sorgenti nei materiali allegati.

T00

Percorso del modulo

3 Dalla domanda al programma

7 Valori, tipi e operazioni

11 Lettura e trasformazione dei dati

18 Errori e controllo del procedimento

20 Preparazione ed esecuzione locale

24 Scelte di strumenti e prestazioni

28 Esecuzione remota e persistenza

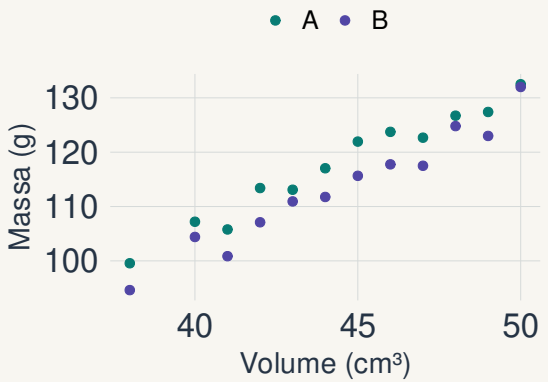
31 Confronti e consegna del lavoro

I numeri indicano le pagine iniziali; i titoli sono cliccabili. Codice, figure e verifiche sono inclusi nel progetto.

Una domanda geologica diventa una procedura

Il corso collega misure, ragionamento statistico e calcolo attraverso procedure delle quali sia possibile ricostruire ogni passaggio. L'introduzione utilizza ventiquattro provini, distribuiti in sei stazioni e due affioramenti, per derivare la densità da massa e volume. Prima di eseguire un comando occorre stabilire il significato delle colonne, le unità e l'oggetto della sintesi. R costituisce il linguaggio principale delle lezioni; gli esercizi Python affiancati permettono di riconoscere quali scelte appartengano al metodo e quali alla sua implementazione.

Evidenza grafica



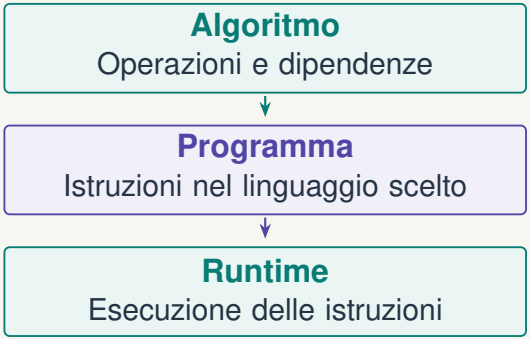
Massa e volume dei ventiquattro provini didattici. Il colore identifica l'affioramento di appartenenza.

Richiami Il programma traduce una domanda definita attraverso variabili, unità e operazioni verificabili.

Algoritmo, linguaggio e programma

Un algoritmo descrive una sequenza di operazioni, mentre un linguaggio fornisce le regole con cui esprimerla in una forma eseguibile. Un programma è una realizzazione di tale procedura; uno script ne costituisce una forma comunemente conservata in un file di testo. L'ambiente di sviluppo offre strumenti per scrivere, avviare e ispezionare il codice, mentre il runtime ne esegue le istruzioni. Questa distinzione permette di comprendere perché lo stesso file R possa essere utilizzato da un terminale oppure da un'interfaccia come RStudio.

Lettura del diagramma

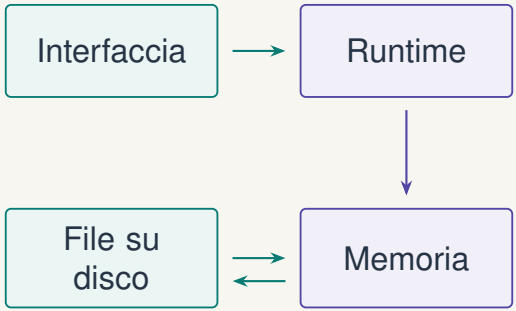


Richiami Il significato della procedura resta distinguibile dagli strumenti con cui viene scritta ed eseguita.

Interfaccia, processo e sistema di calcolo

Quando si avvia una porzione di codice, l'interfaccia la trasmette a un processo che analizza le istruzioni e opera sugli oggetti disponibili. Il processore esegue le operazioni, la memoria conserva temporaneamente dati e stato della sessione, mentre il disco mantiene i file salvati. Le implementazioni dei linguaggi possono tradurre o compilare parti del codice e richiamare librerie native. Per il lavoro iniziale conta soprattutto riconoscere quale processo stia eseguendo il programma e quali file siano stati letti o scritti.

Lettura del diagramma



Richiami File persistenti e oggetti della sessione occupano ruoli diversi nel procedimento di calcolo.

Un nome identifica un oggetto nella sessione

L'assegnazione collega un nome a un oggetto, rendendolo disponibile alle istruzioni successive. La memoria contiene il valore e la struttura dell'oggetto, mentre il nome consente di richiamarlo nel programma. Stampare una tabella produce una rappresentazione leggibile nell'uscita della sessione; per conservarla in un file occorre una scrittura esplicita. Alla chiusura del processo, gli oggetti che esistevano soltanto in memoria possono scomparire, anche se il testo del programma e un output precedentemente salvato restano disponibili.

Lettura del diagramma



Richiami La disponibilità di un output sullo schermo non prova che esista un file contenente il risultato.

La relazione fisica precede la sintassi

La densità del primo provino deriva dal rapporto fra 107,2 grammi e quaranta centimetri cubi, restituendo 2,68 grammi per centimetro cubo. Il codice conserva le unità nei nomi delle variabili e confronta il risultato con un calcolo manuale. In R la freccia assegna un valore, mentre il confronto di uguaglianza usa due segni uguali. In Python l'assegnazione usa un segno uguale e la medesima divisione esprime il rapporto, purché i valori siano numerici e le unità restino coerenti.

$$\rho = \frac{m}{V} = \frac{107{,}2\ \mathrm{g}}{40\ \mathrm{cm}^3} = 2{,}68\ \mathrm{g\,cm}^{-3}$$

Implementazione in R

```
massa_g <- 107.2
volume_cm3 <- 40
rho <- massa_g/volume_cm3
stopifnot(abs(rho-2.68)<1e-12)
rho
```

Richiami Il controllo manuale collega la formula, le unità e l'istruzione eseguita.

Il tipo dell'oggetto determina l'operazione

Un vettore atomico R raccoglie valori di un tipo comune, mentre una tabella può associare colonne di tipi differenti. In Python, una lista e un array numerico interpretano diversamente alcune operazioni: moltiplicare una lista per due ne ripete gli elementi, mentre moltiplicare un array NumPy raddoppia i valori. Il simbolo scritto nel programma acquista pertanto significato attraverso il tipo degli operandi. La verifica di struttura e tipi dopo l'importazione evita che una misura venga trattata come testo o che un codice perda il proprio formato.

Lettura del diagramma

Lista Python

[1, 2] ⟶ [1, 2, 1, 2]

× 2 ripete gli elementi

Array NumPy

[1, 2] ⟶ [2, 4]

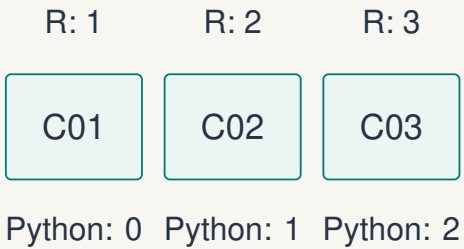
× 2 raddoppia ogni valore

Richiami La lettura del codice richiede di conoscere sia l'operatore sia il tipo degli oggetti coinvolti.

Indici e corrispondenza delle righe

Gli indici numerici ordinari di un vettore R iniziano da uno, mentre quelli di una lista o di un array Python iniziano da zero. Il primo elemento viene quindi richiamato con x[1] in R e x[0] in Python. La differenza riguarda la convenzione di accesso e lascia invariata l'identità dell'osservazione, purché l'ordine dei dati coincida. Negli scambi fra linguaggi conviene verificare gli identificativi, poiché la posizione nella tabella può cambiare dopo ordinamenti, filtri o aggregazioni.

Lettura del diagramma



Stessi provini, convenzioni di accesso differenti.

Richiami La verifica degli identificativi protegge la corrispondenza scientifica oltre la convenzione degli indici.

Operazioni vettoriali e controllo delle dimensioni

Per n provini, la divisione fra i vettori delle masse m e dei volumi V calcola una densità per ogni coppia, mantenendo l'associazione stabilita dall'ordine delle righe. In R vettori di lunghezze differenti possono essere riciclati, producendo talvolta un risultato numericamente plausibile ma privo della corrispondenza desiderata. Il controllo della lunghezza precede quindi l'operazione. Un ciclo esplicito può esprimere lo stesso algoritmo; la scelta fra ciclo e operazione vettoriale deve conservare significato, leggibilità e gestione dei casi limite.

$$\rho_i = m_i / V_i, \qquad i = 1, \ldots, n$$

Implementazione in R

```
massa <- c(10,20,30)
volume <- c(4,8,12)
stopifnot(length(massa)==length(volume),
 all(is.finite(massa)),
 all(is.finite(volume)),all(volume>0))
rho <- massa/volume
stopifnot(all(rho==2.5))
rho
```

Richiami La forma compatta di un'operazione conserva la necessità di verificare l'allineamento delle osservazioni.

Il CSV rappresenta testo organizzato in colonne

Un file CSV contiene caratteri, separatori e righe che il programma interpreta secondo regole di lettura. Il separatore decimale, il simbolo dei mancanti e la codifica dei caratteri incidono sulla ricostruzione della tabella. Un identificativo come 001 deve essere conservato come testo quando gli zeri iniziali fanno parte del codice. Il formato tabellare richiede un dizionario esterno per documentare unità, metodi e significato delle colonne, poiché tali informazioni non sono garantite dalla sola estensione del file.

Lettura del diagramma



Richiami L'importazione interpreta una rappresentazione e richiede convenzioni dichiarate.

Controlli iniziali e cartella di avvio

Il frammento assume che la sessione sia avviata dalla radice del progetto e cerca il CSV nella cartella dati. Dopo la lettura verifica ventiquattro righe, identificativi unici, massa numerica e volumi finiti positivi. L'ispezione della struttura completa questi controlli mostrando i tipi assegnati alle colonne. Se il file risulta assente, si esaminano cartella corrente, nome ed estensione prima di modificare il codice di analisi; un errore di percorso riguarda infatti l'accesso al dato e precede qualsiasi problema statistico.

Implementazione in R

```
f <- file.path('dati','campioni_modulo1.csv'
    )
stopifnot(file.exists(f))
d <- read.csv(f,stringsAsFactors=FALSE)
stopifnot(nrow(d)==24,!anyDuplicated(d$id),
 is.numeric(d$massa_g),
 all(is.finite(d$volume_cm3)),
 all(d$volume_cm3>0))
str(d)
```

Richiami Il controllo dell'input distingue problemi di accesso, struttura e contenuto prima del calcolo.

Filtrare conservando l’unità della riga

Un filtro seleziona righe intere, preservando l’associazione fra identificativo id, massa m, volume V e stazione s. Indichiamo con A l’insieme degli indici che soddisfano la condizione e con D_A la tabella risultante. Nell’esempio si conservano i provini con volume almeno pari a 45 centimetri cubi. Ordinare separatamente le colonne distruggerebbe la corrispondenza fra misure dello stesso provino; il confronto degli identificativi e il conteggio delle righe permettono invece di verificare l’effetto della selezione.

$$D_A = \{(id_i, m_i, V_i, s_i) : i \in A\}$$

Lettura del diagramma

Condizione: $V \geq 45\ \mathrm{cm}^3$

C01 · A1 · 107,2 g · 40 cm³
escluso

C02 · A1 · 121,95 g · 45 cm³
incluso

C03 · A1 · 132,5 g · 50 cm³
incluso

$A = \{2, 3\}$: due righe complete

Richiami L’identità del provino resta il riferimento di ogni trasformazione della tabella.

Valore mancante e osservazione censurata

Un valore registrato come inferiore a 0,05 informa che la misura ricade sotto un limite, mentre un valore mancante segnala l'assenza dell'informazione secondo il protocollo adottato. La sostituzione automatica con zero cancellerebbe questa distinzione e introdurrebbe un valore esatto non osservato. Il dataset analitico dovrebbe conservare valore, limite e indicatore di censura in campi distinti. Nei calcoli introduttivi, la gestione di NA viene resa esplicita e collegata al numero di osservazioni effettivamente utilizzate.

Sviluppo e interpretazione

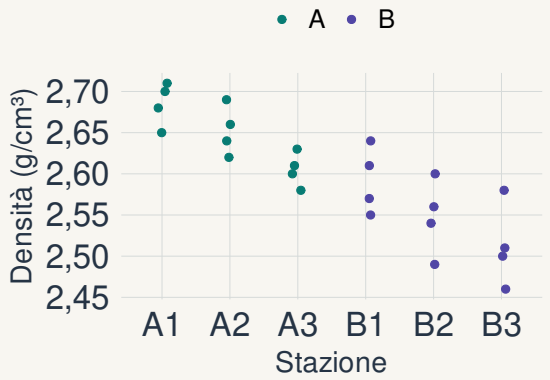
Per x = c(1, NA, 3), mean(x) restituisce NA, mentre mean(x, na.rm = TRUE) restituisce 2 sui due valori disponibili. L'esclusione è computazionalmente definita; la sua pertinenza dipende dal motivo per cui manca la misura e dalla popolazione che si vuole descrivere.

Richiami Codici di assenza e limiti di misura conservano informazioni diverse sul processo osservativo.

La variabile derivata conserva le osservazioni

La nuova colonna di densità viene calcolata per ciascun provino, lasciando disponibili massa e volume originari. Il grafico per stazione permette di riconoscere il raggruppamento dei dati e la variabilità fra provini dello stesso luogo. La lieve separazione orizzontale dei punti facilita la lettura senza modificare la densità rappresentata sull'asse verticale. Prima di sintetizzare le differenze fra affioramenti occorre mantenere evidente che le ventiquattro righe appartengono a sei unità geografiche distinte.

Evidenza grafica



Densità individuali ordinate per stazione. Il colore mantiene l'affioramento e rende visibile la struttura del campione.

Richiami Una variabile derivata si interpreta insieme alle misure originarie e al disegno di campionamento.

Media dei rapporti e rapporto delle somme

Per n provini, con masse m_i e volumi V_i, la media delle densità attribuisce uguale peso a ciascun provino; il rapporto fra massa totale e volume totale assegna invece pesi w_i proporzionali ai volumi. Le due quantità valgono qui circa 2,59917 e 2,59984 grammi per centimetro cubo. La vicinanza numerica dipende da questi dati, mentre la diversa ponderazione corrisponde a due distinti significati fisici.

$$\overline{\rho} = \frac{1}{n}\sum_i \frac{m_i}{V_i}, \qquad \rho_{tot} = \frac{\sum_i m_i}{\sum_i V_i}$$

$$\rho_{tot} = \sum_i w_i \rho_i, \qquad w_i = \frac{V_i}{\sum_j V_j}$$

Sviluppo e interpretazione

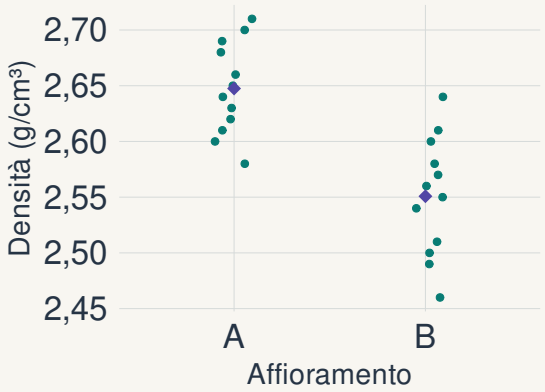
La prima sintesi descrive il provino medio secondo pesi uguali. La seconda descrive il rapporto complessivo del materiale considerato, sotto una somma di volumi pertinente al problema. La scelta deve precedere l'interpretazione delle differenze fra gruppi.

Richiami L'aggregazione incorpora una ponderazione che deve essere coerente con la domanda fisica.

Sintesi per affioramento e livello dell’inferenza

Le medie dei dodici provini dell’affioramento A e dei dodici dell’affioramento B sono rispettivamente 2,6475 e circa 2,55083 grammi per centimetro cubo. La figura conserva i valori individuali affinché la sintesi resti collegata alla dispersione osservata. Queste medie descrivono il campione disponibile; un confronto inferenziale richiederà di considerare le tre stazioni per affioramento, la dipendenza dei provini e il modo in cui i luoghi sono stati selezionati. Il calcolo corretto costituisce dunque un passaggio della costruzione dell’argomento scientifico.

Evidenza grafica



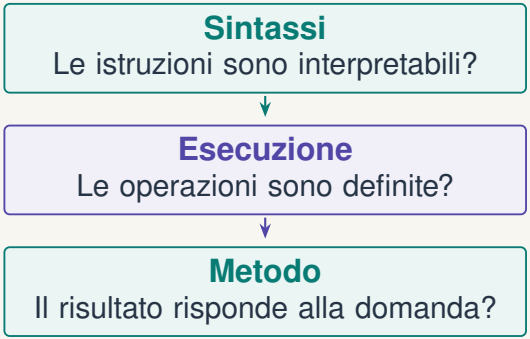
Punti verdi: densità individuali. Rombi viola: medie per affioramento sui provini disponibili.

Richiami La sintesi descrittiva conserva un dominio osservativo che l’inferenza deve esplicitare.

Localizzare un errore attraverso il passaggio

Un errore di sintassi impedisce di interpretare correttamente le istruzioni; un errore durante l'esecuzione interrompe un'operazione; un errore metodologico può produrre un risultato plausibile e completare il programma. La diagnosi segue quindi la sequenza dell'analisi, verificando file letto, struttura, condizioni del calcolo e significato della sintesi. Un esempio ridotto facilita l'identificazione del passaggio responsabile. Dopo la correzione si ripete l'esecuzione completa da una sessione pulita, per verificare che il risultato dipenda soltanto dagli input dichiarati.

Lettura del diagramma



Richiami La riuscita tecnica dell'esecuzione deve essere accompagnata dall'esame della pertinenza del metodo.

Funzioni piccole e valori di riferimento

Una funzione raccoglie un'operazione definita attraverso argomenti e risultato, rendendo possibile verificarla separatamente dal resto dell'analisi. La funzione di densità richiede volumi positivi e associa masse e volumi della stessa lunghezza. La divisione di 107,2 g per 40 cm³ deve restituire 2,68 g/cm³; un volume nullo verifica il comportamento al limite del dominio. Separare importazione, trasformazione e scrittura permette di riutilizzare il calcolo senza incorporare dipendenze nascoste da file o oggetti della sessione.

$$f(m, V) = m/V, \qquad V > 0$$

Sviluppo e interpretazione

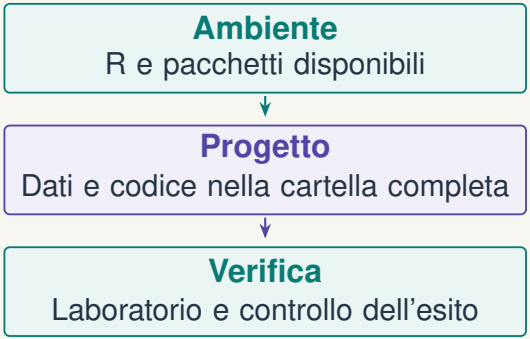
Un controllo utile confronta il risultato con un valore noto oppure verifica una proprietà della trasformazione. Conservare gli identificativi dopo l'aggiunta di una colonna è un esempio di invariante; confrontare un'espressione con una copia identica della stessa espressione offre invece poca evidenza indipendente.

Richiami Una funzione verificabile rende espliciti input, dominio e risultato atteso.

Preparare runtime, interfaccia e progetto

L'esecuzione locale richiede un'installazione di R compatibile con il sistema operativo, i pacchetti utilizzati e una copia completa del progetto. Le distribuzioni ufficiali sono disponibili attraverso CRAN, con istruzioni specifiche per Windows, macOS e Linux. RStudio può essere aggiunto come interfaccia di lavoro. Dopo l'estrazione dell'archivio si apre il progetto dalla cartella principale e si verifica la lettura del primo CSV, così da separare i problemi dell'ambiente da quelli introdotti successivamente dall'analisi.

Lettura del diagramma



Richiami Il controllo iniziale dell'ambiente riduce l'ambiguità degli errori incontrati durante l'analisi.

Avviare un file e controllarne l’esito

Dalla radice del progetto, un terminale può eseguire il primo laboratorio mediante Rscript laboratori/T00/T00_N01.R. Nell’ambiente di sviluppo si apre il medesimo file e lo si esegue dall’inizio, verificando messaggi e risultati. Il percorso Python usa python laboratori_python/T00/T00_N01.py, con un interprete nel quale siano disponibili le dipendenze dichiarate. L’esito comprende codice di terminazione, output prodotto e controlli superati; la semplice comparsa di un grafico non garantisce che tutte le istruzioni siano state completate.

Sviluppo e interpretazione

Prima dell’esecuzione, getwd() mostra la cartella corrente in R. Il progetto include dati, funzioni di supporto, script e destinazioni dei risultati. Il trasferimento di un solo file può interrompere queste dipendenze, anche quando il codice rimane leggibile.

Richiami L’istruzione di avvio deve dichiarare cartella iniziale, file da eseguire e risultato da controllare.

Installazione, caricamento e ambiente attivo

L'installazione rende un pacchetto disponibile in una libreria, mentre il caricamento lo rende utilizzabile nella sessione secondo le convenzioni del linguaggio. In R, library carica un pacchetto già presente; la forma pacchetto::funzione richiama esplicitamente una funzione esportata. In Python, l'ambiente virtuale separa le dipendenze del progetto e viene creato dall'interprete selezionato. Quando un pacchetto risulta installato ma non viene trovato, si confrontano ambiente attivo, percorsi delle librerie e processo utilizzato dall'interfaccia.

Sviluppo e interpretazione

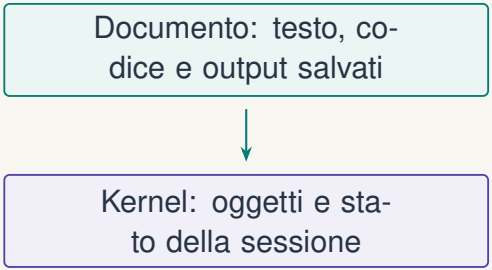
La preparazione delle dipendenze precede gli script analitici. Per Python, python -m venv .venv crea un ambiente dall'interprete richiamato; python -m pip collega l'installazione a quell'interprete. In R, sessionInfo() e .libPaths() documentano ambiente e librerie utilizzate.

Richiami La disponibilità di un pacchetto va verificata nell'ambiente che esegue effettivamente il codice.

Notebook, ordine delle celle e stato della sessione

Un notebook conserva testo, codice e output, mentre il kernel mantiene gli oggetti della sessione di esecuzione. Una cella avviata fuori ordine può utilizzare valori creati in precedenza e produrre un risultato che il documento, letto dall'inizio, non ricostruisce. Il controllo consiste nel riavviare il processo ed eseguire tutte le celle in ordine, verificando dipendenze e risultati. Un output salvato nel documento testimonia una precedente esecuzione, della quale occorre distinguere il contenuto dallo stato attualmente disponibile in memoria.

Lettura del diagramma



Riavvio ed esecuzione ordinata verificano le dipendenze.

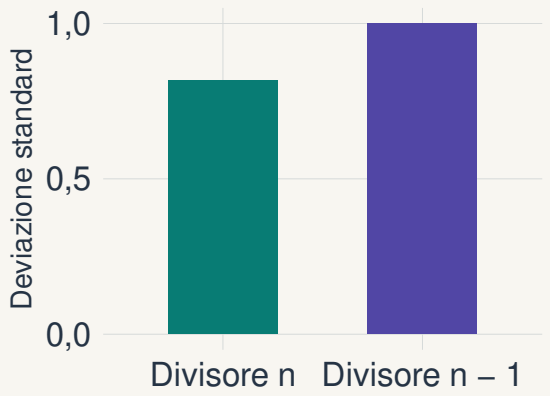
Richiami La verifica da una sessione pulita collega il documento salvato al procedimento effettivamente eseguibile.

Convenzioni statistiche e confronto fra linguaggi

Due implementazioni possono restituire risultati differenti perché adottano convenzioni diverse, pur applicando entrambe formule corrette. Per i tre valori x_i = 1, 2, 3, con n = 3 e media pari a 2, la deviazione standard con divisore n vale circa 0,8165, mentre quella con divisore n meno uno vale uno. R sd usa il secondo criterio; NumPy richiede ddof uguale a uno per ottenere lo stesso risultato. Il confronto deve perciò dichiarare formula, gestione dei mancanti, ordine delle righe e parametri, prima di attribuire una differenza al linguaggio.

$$s = \sqrt{\frac{\sum_i (x_i - \overline{x})^2}{n-1}}, \qquad \sigma_n = \sqrt{\frac{\sum_i (x_i - \overline{x})^2}{n}}$$

Evidenza grafica



Deviazioni standard dei valori 1, 2, 3 secondo due divisori. Il confronto usa gli stessi dati.

Richiami L'equivalenza metodologica richiede convenzioni esplicite oltre alla corrispondenza della sintassi.

Ecosistemi e criteri di scelta

R offre un ambiente dedicato al calcolo statistico, nel quale formule dei modelli e rappresentazioni grafiche costituiscono elementi centrali del lavoro. Python collega analisi numerica e attività generaliste attraverso librerie e strumenti differenti. La scelta per un progetto dipende da competenze del gruppo, metodi necessari, integrazione con altri sistemi e manutenzione del codice. Nel corso, R mantiene continuità con le lezioni statistiche, mentre il confronto Python serve a distinguere le scelte analitiche dalle convenzioni delle librerie utilizzate.

Sviluppo e interpretazione

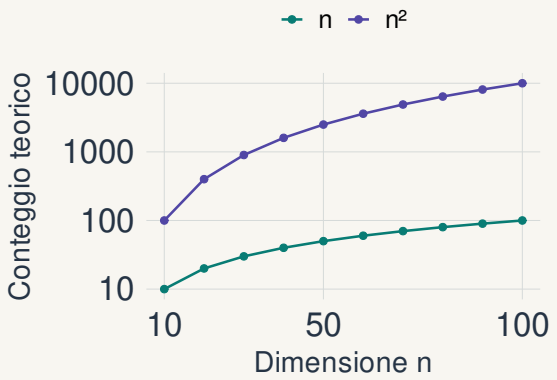
Il riciclo dei vettori, la selezione delle colonne, la gestione degli indici e i valori predefiniti richiedono attenzione in entrambi gli ecosistemi. La documentazione della funzione utilizzata e un esempio verificabile offrono un riferimento più utile di una graduatoria generale fra linguaggi.

Richiami Lo strumento viene scelto rispetto al compito, alle dipendenze e alle competenze disponibili.

Costo dell’algoritmo e dimensione del problema

Il tempo di esecuzione dipende da algoritmo, dimensione dei dati, librerie native, memoria e organizzazione del calcolo. La figura confronta due conteggi teorici, proporzionali a n e a n al quadrato, per mostrare come un aumento della dimensione possa amplificare differenze algoritmiche. I valori rappresentano operazioni ipotetiche e non tempi misurati. Un confronto fra implementazioni richiede lo stesso compito, dati equivalenti, risultati verificati e misure ripetute, separando eventuali costi di caricamento o inizializzazione.

Evidenza grafica



Conteggi teorici n e n² su asse verticale logaritmico. La figura illustra la crescita, senza costituire un benchmark.

Richiami La prestazione computazionale deve essere misurata su un compito definito e con risultati equivalenti.

Memoria, parallelismo e trasferimenti

Una matrice densa con n righe e p colonne di numeri a doppia precisione richiede circa otto byte per elemento, ai quali si aggiungono struttura dell'oggetto e copie temporanee. Operazioni che creano matrici intermedie possono quindi richiedere più memoria del solo input. Il calcolo parallelo distribuisce parti del lavoro, introducendo costi di coordinamento e trasferimento. Un processore grafico (GPU) accelera compiti per i quali esistano implementazioni appropriate e una quantità di lavoro sufficiente; la sua presenza non modifica automaticamente il codice eseguito sul processore centrale (CPU).

$M_{valori} \simeq 8np\ \mathrm{byte}$ per una matrice double $n \times p$

Sviluppo e interpretazione

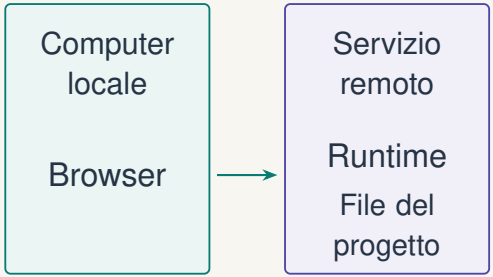
La misura della memoria deve includere gli oggetti intermedi effettivamente creati. Per ridurre i costi può essere più efficace cambiare rappresentazione, evitare copie o riformulare l'algoritmo, prima di distribuire il lavoro su più processi.

Richiami Risorse e parallelismo vanno valutati rispetto alla struttura delle operazioni e dei dati.

Il browser può controllare un processo remoto

Un’interfaccia aperta nel browser può comunicare con un runtime sullo stesso computer oppure con un processo ospitato da un servizio remoto. Nel secondo caso, il codice opera sui file e sulle librerie disponibili nell’ambiente remoto, che devono essere predisposti anche quando il documento è visibile localmente. La connessione collega interfaccia e processo, mentre la persistenza dei file dipende dal servizio e dalla destinazione di salvataggio. La distinzione permette di interpretare correttamente la perdita di oggetti dopo un riavvio del runtime.

Lettura del diagramma



Il luogo del calcolo determina file e ambiente disponibili.

Richiami Il processo che esegue il codice determina quali risorse siano effettivamente accessibili.

Progetti R, notebook gestiti e servizi istituzionali

Un progetto RStudio in Posit Cloud raccoglie codice, pacchetti e dati entro un ambiente remoto; Colab offre un servizio di notebook Jupyter con runtime gestito. Un JupyterHub istituzionale dipende dalla configurazione predisposta dall'ateneo, mentre Binder costruisce ambienti dimostrativi a partire da repository configurati. Per il corso si trasferisce il progetto completo e si verifica l'ambiente prima dell'esecuzione. Risorse, durata delle sessioni e condizioni d'uso possono variare, per cui la scelta operativa si riferisce alla documentazione del servizio e al piano effettivamente disponibile.

Sviluppo e interpretazione

Per Posit Cloud si crea un progetto RStudio, si caricano i file e si avvia il laboratorio dalla radice. Per un notebook Python si rendono disponibili anche dati e moduli di supporto, si verifica l'interprete del kernel e si esegue il documento dall'inizio.

Richiami La disponibilità dell'interfaccia va accompagnata dalla verifica dell'ambiente e delle dipendenze del progetto.

Recuperare i risultati prima della fine della sessione

Un file creato nel disco temporaneo di un runtime può essere eliminato alla cessazione della sessione, mentre il notebook salvato può conservare ancora la rappresentazione dell'output. La procedura conclude quindi l'analisi esportando tabelle e figure in una destinazione persistente e verificandone la rilettura. Il laboratorio T00_N20 simula localmente questa situazione, copiando un risultato prima di eliminare la cartella temporanea. Nelle attività reali si controllano inoltre autorizzazioni, condizioni di utilizzo dei dati e contenuto effettivo della consegna.

Lettura del diagramma



Richiami La conclusione del calcolo comprende la conservazione verificata dei prodotti necessari.

Identità delle estrazioni e delle rappresentazioni

Lo stesso seme può inizializzare sequenze diverse in R e Python, poiché generatore e implementazione delle distribuzioni fanno parte del procedimento. Per confrontare simulazioni si possono conservare estrazioni comuni e trasformarle secondo convenzioni dichiarate. Anche rappresentazioni equivalenti possono differire: il segno di una componente principale è arbitrario, mentre le etichette numeriche dei gruppi possono essere permutate. Il confronto deve individuare la proprietà scientifica da conservare, anziché richiedere identità formale a ogni elemento dell'output.

Sviluppo e interpretazione

Supponiamo che tre provini ricevano le etichette di gruppo (1, 1, 2) in R e (2, 2, 1) in Python. In entrambe le analisi i primi due provini appartengono allo stesso gruppo e il terzo a un gruppo distinto: è cambiato il nome dei gruppi, mentre la suddivisione è identica. Il confronto deve quindi riguardare le appartenenze, tenendo conto delle possibili permutazioni delle etichette.

Richiami Il criterio di equivalenza va definito rispetto alla quantità scientifica rappresentata.

Dal file iniziale a una tabella rileggibile

La dimostrazione riunisce lettura, trasformazione, raggruppamento e scrittura, conservando una verifica numerica della tabella esportata. Il file temporaneo permette di esercitare l'intero passaggio senza modificare i dati originari. Le densità medie attese sono 2,6475 g/cm³ per A e circa 2,55083 g/cm³ per B; gli identificativi verificano il significato delle righe. La consegna applicata utilizza una destinazione persistente e include codice modificabile, dati distribuibili, provenienza, istruzioni di avvio e registri delle verifiche eseguite.

Implementazione in R

```
d <- read.csv('dati/campioni_modulo1.csv')
d$rho <- d$massa_g/d$volume_cm3
s <- aggregate(rho~affioramento,d,mean)
f <- tempfile(fileext='.csv')
write.csv(s,f,row.names=FALSE)
q <- read.csv(f)
stopifnot(identical(q$affioramento,s$
    affioramento),
 max(abs(q$rho-s$rho))<1e-12)
unlink(f)
s
```

Richiami La rilettura del prodotto esportato collega il risultato analitico al file effettivamente consegnato.

Studio autonomo e continuità con i moduli

I venti laboratori introduttivi sono disponibili in R e Python e sviluppano gli stessi oggetti di calcolo: valori, tipi, indici, lettura, filtri, densità, sintesi, errori, convenzioni e persistenza. Lo studio autonomo richiede di spiegare ogni passaggio prima di modificarlo, prevedendo il risultato di un esempio semplice e confrontandolo con l'esecuzione. M01 approfondisce misura e campionamento, M02 l'organizzazione del lavoro in R e M15 la riproducibilità. I moduli successivi riprendono queste basi nella costruzione degli argomenti statistici.

Sviluppo e interpretazione

La verifica conclusiva ricostruisce il percorso del dato: provenienza del file, significato delle colonne, oggetto creato in memoria, trasformazione applicata e risultato salvato. Una modifica viene accompagnata da motivazione, risultato atteso e controllo dell'effetto osservato.

Richiami La comprensione del procedimento consente di trasferire il codice a problemi nuovi conservandone il significato.